Laboratorio 2.2 — Pipeline de observabilidad con OpenTelemetry

Informe técnico de la implementación propia en `Lab 2.2`. Las cifras de este documento salen de la corrida local del 9 de octubre de 2026. Los despliegues en GKE y ECS están preparados y no se ejecutaron: este workspace no tenía credenciales de GCP ni de AWS, y tampoco el binario de Terraform.

1. Introducción y objetivos

El laboratorio pide un sistema distribuido que produzca métricas, logs y trazas de una misma operación de negocio, y un pipeline que las lleve hasta herramientas donde se puedan consultar juntas. El repositorio del docente (`oTelLabs-main`, Python y FastAPI) sirvió para entender el problema. No es la solución entregada y no se modificó.

La implementación propia usa dos microservicios Spring Boot 3.5.16 sobre Java 21:

- `service-a` consulta un pedido en MySQL y llama a `service-b`.
- `service-b` consulta el inventario de ese SKU.

La operación de demostración es `GET /api/orders/{orderId}`. La respuesta dice si el stock alcanza (`available >= quantity`). Los pedidos `ord-001` a `ord-005` se cargan al crear la base.

Objetivos cubiertos en local:

- Trazas distribuidas desde la petición HTTP hasta JDBC y la llamada interna.
- Métricas HTTP, JVM y de negocio en Prometheus, con un dashboard de Grafana.
- Logs JSON con `trace_id` y `span_id`, consultables en Loki y enlazados a Tempo.
- OpenTelemetry Collector 0.162.0 como único punto de salida de las tres señales.
- Comparación de carga, con y sin el agente Java, sobre la misma lógica de negocio.

Objetivos dejados listos, sin apply:

- GKE con Cloud Trace, Cloud Logging y Managed Prometheus.
- ECS Fargate con X-Ray, CloudWatch Logs y EMF.

2. Arquitectura

```
Cliente
  |  HTTP :8080
  v
service-a  --JDBC--> MySQL :3306
  |  HTTP :8081  (traceparent W3C)
  v
service-b  --JDBC--> MySQL
  |
  |  OTLP gRPC :4317   (trazas, métricas, logs)
  v
OpenTelemetry Collector
  |-- otlp_grpc/tempo --> Tempo --> Grafana
  |-- prometheus :8889 --> Prometheus --> Grafana
  +-- otlp_http/loki  --> Loki --> Grafana
```

MySQL 8.4 guarda `orders` e `inventory`. El acceso es `JdbcTemplate`, no JPA. El agente Java instrumenta el driver; un span manual de SQL duplicaría esa información. Los spans manuales describen la regla de negocio: `order.compose` e `inventory.lookup`.

El agente es `opentelemetry-javaagent` 2.31.1. La aplicación solo depende de la API 1.65.0. El SDK de runtime lo aporta el agente, así que no hay dos SDK exportando a la vez. `OTEL_ENABLED=false` arranca el mismo jar sin `-javaagent`. Esa es la diferencia entre el baseline y el escenario instrumentado.

En local el muestreo es `parentbased_traceidratio` con argumento `1.0`: se conserva cada traza. El intervalo de métricas es 15 segundos. Los propagadores son `tracecontext` y `baggage`.

GCP y AWS no añaden un segundo diseño de aplicación. Cambian el exporter del Collector y la plataforma que ejecuta los contenedores. Los YAML están en `observability/collector/collector-gcp.yaml` y `collector-aws.yaml`. Terraform está en `terraform/gcp` y `terraform/aws`.

3. Decisiones de diseño

Decisión	Motivo
Java y Spring Boot	Es el stack de mantenimiento pedido. El ejemplo del docente es Python.
Agente Java, no SDK embebido	La auto-instrumentación de HTTP y JDBC ya cubre el camino técnico. El código de negocio solo abre spans de dominio.
Tempo, sin Jaeger	Grafana ya consulta Tempo. Un segundo almacén de trazas no aporta una señal nueva en local.
Loki para logs locales	Hace falta un backend que Grafana pueda filtrar por `trace_id`. Cloud Logging queda para GKE.
Prometheus hace scrape del Collector (`:8889`)	Las aplicaciones no exponen `/metrics`. Evita un segundo registro Micrometer.
MySQL en el compose y, en ECS, como sidecar opcional	La traza tiene que mostrar persistencia. RDS queda detrás de `create_rds=false` para no crear una base de pago sin una decisión explícita.
Sin `googlecloud` en el compose local	Ese exporter exige credenciales y un proyecto. En el ejemplo del docente quedaba activo con una ruta de ADC de otra máquina.

El ejemplo del docente incrementaba `http_requests_total` dos veces por petición exitosa, así que un SLI de disponibilidad caía cerca del 50 % sin errores reales. Aquí el histograma HTTP lo emite el agente una vez por petición. El dashboard filtra `http_route=~"/api/.*"`. El patrón de exclusión del agente es `.*/actuator/.*`: un patrón que solo empieza por `/actuator/` no coincide con la URL completa y el healthcheck contaminaba las series.

Los exporters `otlp` y `otlphttp` están deprecados en el Collector 0.162.0. La configuración local usa `otlp_grpc/tempo` y `otlp_http/loki`. Los tres YAML pasaron `otelcol-contrib validate`.

4. Los tres pilares

Trazas

El agente abre el span servidor `GET /api/orders/{orderId}`. `OrderFulfillmentService` abre `order.compose` como hijo, con `order.id` y `product.sku`. El `SELECT` de pedidos aparece como span JDBC. `RestClient` propaga

`traceparent`. En service-b el span servidor es `GET /api/inventory/{sku}`, el de negocio es `inventory.lookup` y el de base de datos es `SELECT lab22.inventory`.

Un 404 de pedido no marca el span de negocio como error: el cliente pidió un id que no existe. Un fallo de service-b sí marca `order.compose` en error. Eso está cubierto por las pruebas de `mvn test`.

Métricas

El agente exporta `http.server.request.duration` y métricas JVM. La aplicación suma contadores de negocio (`orders.composed`, consultas y reservas de inventario). El Collector los publica en `:8889` con `UnderscoreEscapingWithSuffixes`. El nombre que Prometheus scrapea para la latencia HTTP es `http_server_request_duration_seconds_count`. La CPU del proceso es `jvm_cpu_recent_utilization_ratio`. El nombre sin `_ratio` no existe en esta traducción y dejaba el panel vacío.

Logs

Logback escribe JSON a stdout con `trace_id`, `span_id`, servicio, versión y entorno. El agente reenvía el mismo evento por OTLP. El processor `transform/logs` antepone `trace_id=<32 hex>` al cuerpo. Grafana enlaza ese texto con la expresión `trace_id=([0-9a-f]{32})`. En Loki el identificador es structured metadata, no etiqueta de stream: `{service_name="service-a"} | trace_id="..."` encuentra la línea; seleccionar `trace_id` como etiqueta de índice no es el camino correcto.

5. Collector

Receivers: OTLP gRPC `4317` y HTTP `4318`.

Orden de processors en trazas: `memory_limiter` (400 MiB, pico de 100), `resource`, `filter/health`, `batch` (5 s, 1024, tope 2048). El limitador va primero para rechazar antes de gastar memoria. El batch va al final para no empaquetar spans que el filtro va a tirar. Los logs pasan además por `transform/logs`.

Salud: extensión `health_check` en `:13133` y métricas internas en `:8888`. Prometheus scrapea ese puerto con el job `otel-collector`. El panel de fallos usa `or vector(0)` porque las series de error no existen hasta que hay un fallo.

En GCP el Collector exporta trazas y logs con `googlecloud` y métricas con `googlemanagedprometheus`. La cuenta de servicio prevista tiene `cloudtrace.agent`, `logging.logWriter` y `monitoring.metricWriter`. En AWS exporta con `awsxray`, `awscloudwatchlogs` y `awsemf`. La imagen de Fargate hornea `collector-aws.yaml` porque la task no tiene un bind mount.

6. Evidencia de una petición real

El 9 de octubre de 2026, con `OTEL_ENABLED=true` después del benchmark, `GET /api/orders/ord-001` respondió:

- pedido `ord-001`, SKU `LAPTOP-X1`, cantidad 1, estado `delivered`, cliente `cust-100`;
- inventario: 15 disponibles, 2 reservadas, almacén `WH-BOGOTA`;
- `stockSufficient=true`;
- `traceId=77fa223e7fd82dc6fb4ca2aea30f7f22`.

El stdout de service-a, en el mismo segundo (`2026-10-09T20:34:21.102Z`):

```
{"level":"INFO","message":"Pedido compuesto orderId=ord-001 sku=LAPTOP-X1
sufficient=true","span_id":"6f4e7346db5b3b78","trace_id":"77fa223e7fd82dc6fb4ca2aea30f7f22","service":"service-
a"}
```

El stdout de service-b (`20:34:21.100Z`):

```
{"level":"INFO","message":"Inventario consultado sku=LAPTOP-X1
available=15","span_id":"e971b873e68f1ccf","trace_id":"77fa223e7fd82dc6fb4ca2aea30f7f22","service":"service-
b"}
```

El `trace_id` coincide. Los `span_id` no: cada log se emitió dentro de su span de negocio.

Loki devolvió dos líneas cuyo cuerpo empieza por `trace_id=77fa223e7fd82dc6fb4ca2aea30f7f22`, una por servicio. La consulta de structured metadata `{service_name=~"service-a|service-b"} | trace_id="77fa223e7fd82dc6fb4ca2aea30f7f22"` también devolvió esas dos líneas. La primera consulta, hecha a los pocos segundos, devolvió cero: el log ya estaba en stdout y todavía no en Loki.

Tempo, en la segunda lectura del mismo id, devolvió 13 spans:

Servicio	Span
service-a	`GET /api/orders/{orderId}`, `order.compose`, `SELECT lab22.orders`, `GET` (cliente)
service-b	`GET /api/inventory/{sku}`, `inventory.lookup`, `SELECT lab22.inventory`

También aparecen `Ping`, `Prepare statement` y `Execute prepared statement`. Son del agente JDBC, no spans manuales duplicados. La primera lectura por id solo traía service-a. Unos segundos después estaban los dos servicios. Es la carrera del ingester de Tempo: la búsqueda o una relectura encuentran la traza completa. `tests/verify-local.ps1` reintenta por eso.

Prometheus, ventana de 30 minutos en esa misma sesión:

- SLI de disponibilidad de rutas `/api/`: `100`;
- tasa de errores 5xx: `0`;
- `up{job="otel-collector"}`: `1`.

Los archivos están en `docs/evidencias/`.

7. Dashboard

El dashboard aprovisionado es «Lab 2.2 — Pedidos e inventario» (`uid lab22-observability`). La captura `docs/evidencias/grafana-dashboard.png` corresponde a los últimos 30 minutos de esa sesión, después de la carga de k6.

Panel	Qué se vio
1. Tasa HTTP	Pico cercano a 26 req/s en service-a y service-b alrededor de las 15:30, hora local, que es la ventana del benchmark.
2. Latencia p95 y p99	El gráfico marca picos de hasta unos 2 s. No es el p99 de k6. `histogram_quantile` sobre `rate()` se dispara cuando el intervalo tiene pocas muestras y un máximo alto. El percentil de la prueba de carga está en la sección 8.
3. Errores 5xx	Línea en 0 % para los dos servicios. Antes, sin serie 5xx, Grafana decía "No data". La consulta añade un cero derivado del total.
4. SLI	Gauge en 100 %.
5. CPU y memoria	Tras la carga, en reposo: service-a CPU reciente 0,000277 y heap 73 064 368 bytes; service-b 0,000222 y 81 810 160 bytes. Es `jvm_cpu_recent_utilization_ratio`, no el porcentaje de `docker stats` durante k6.
6. Collector	El job `otel-collector` estaba en 1. El panel de spans exportados y fallos comparte esa captura; los fallos se dibujan en 0 cuando la serie de error no existe.

Navegación configurada y comprobada en la API de Grafana después de reiniciar:

- Loki, derived field `TraceID`, regex `trace_id=([0-9a-f]{32})`, destino Tempo, URL `${__value.raw}`.
- Tempo, `tracesToLogsV2`, consulta `{service_name=~"service-a|service-b"} | trace_id=` más el id de la traza.
- Prometheus, exemplars hacia Tempo.

El archivo de aprovisionamiento escribe `$$` delante de esas variables. Con un solo `$`, Grafana las borra al leer el YAML porque cree que son variables de entorno. La API llegó a devolver la consulta de logs con el id vacío. Tras el escape, la API devuelve de nuevo `${__trace.traceId}` y `${__value.raw}`. El clic del exemplar en la interfaz no se ejercitó; el enlace por `trace_id` en el log y la consulta LogQL sí.

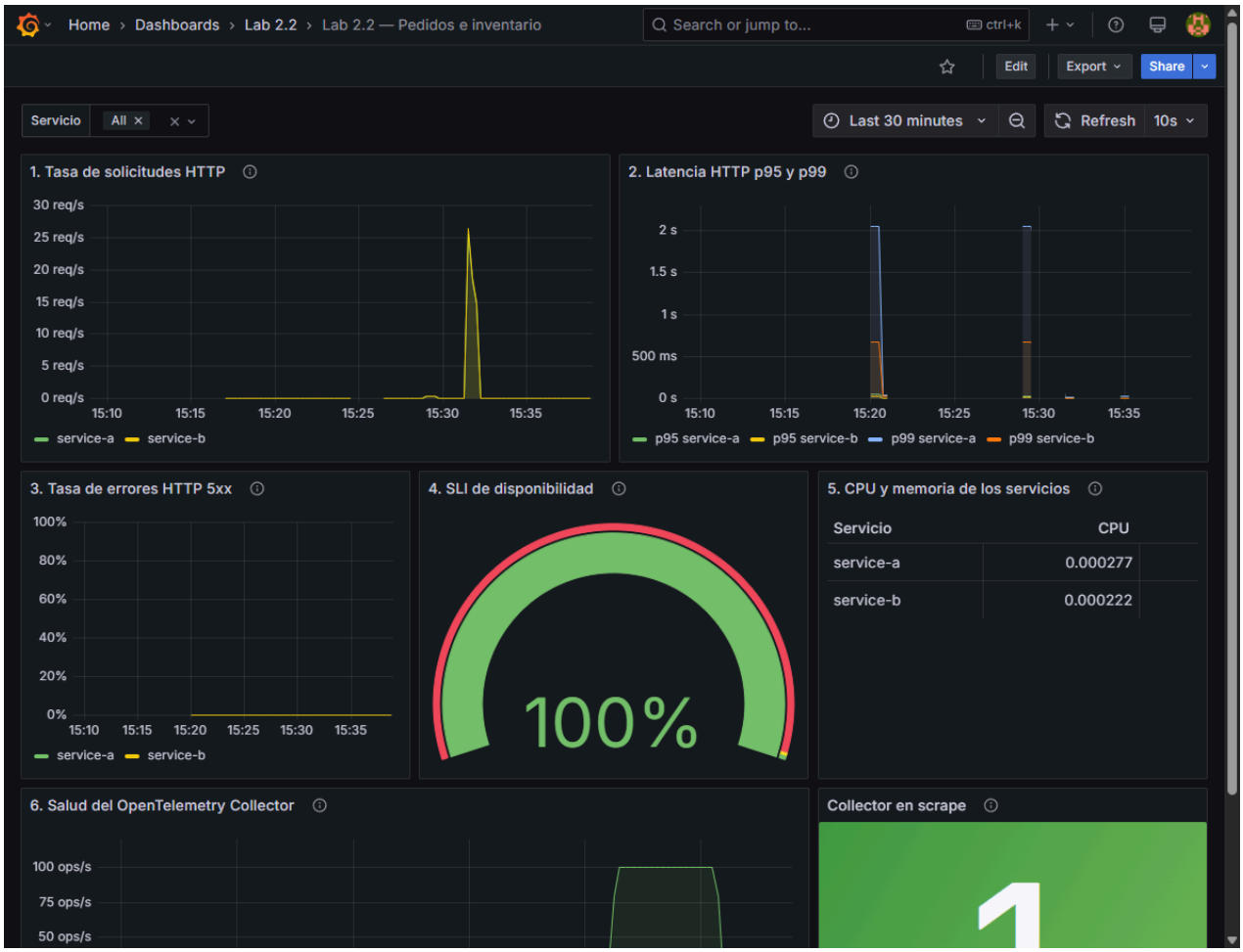


Figura 1. Dashboard local después del benchmark. El SLI marca 100 % y la tasa de errores 5xx está en cero.

8. Benchmark

Script: `load-tests/run-benchmark.ps1`. k6 0.57.0 en Docker, 5 usuarios virtuales, 30 segundos, pausa de 100 ms, pedidos `ord-001` a `ord-005`, URL `http://host.docker.internal:8080`. Misma imagen, misma base y mismo Compose. Única variable de aplicación: `OTEL_ENABLED`. El Collector siguió encendido en el baseline, pero el proceso Java no cargaba el agente. Apagar solo el Collector, como hace el README del ejemplo, dejaría el SDK creando spans y no mediría el costo de instrumentar.

`docker stats` muestreó service-a, service-b y el Collector durante 40 segundos en cada pasada. Fórmulas, aplicadas por `load-tests/analyze_overhead.py`:

- overhead absoluto = instrumentado − baseline
- overhead porcentual = (instrumentado − baseline) / baseline × 100

Métrica	Baseline	Instrumentado	Absoluto	%
Latencia p50 (ms)	8,6306	8,6653	0,0347	0,40 %
Latencia p95 (ms)	14,7352	16,4087	1,6735	11,36 %
Latencia p99 (ms)	21,6424	23,8405	2,1981	10,16 %
Throughput (req/s)	44,2853	43,2927	−0,9926	−2,24 %
Solicitudes	1331	1302	−29	−2,18 %
Tasa de errores	0	0	0	no aplica

Contenedor	CPU base	CPU OTel	CPU %	Mem base	Mem OTel	Mem %
service-a	23,98 %	34,28 %	+42,97 %	293,04 MiB	403,50 MiB	+37,69 %
service-b	18,53 %	33,49 %	+80,69 %	266,23 MiB	431,27 MiB	+61,99 %
collector	1,58 %	0,91 %	−42,41 %	57,25 MiB	70,61 MiB	+23,33 %

Checks de k6: 5320/5320 en baseline y 5204/5204 instrumentado. En el baseline el check de `traceId` no mira el cuerpo. En el instrumentado exige 32 hexadecimales distintos de cero, y pasó.

9. Lectura del overhead

El p50 sube 0,03 ms. En una sola pasada de 30 segundos eso es ruido. El p95 y el p99 suben 1,7 ms y 2,2 ms, cerca del 11 % y del 10 %. El caudal baja un 2 %. No hubo errores HTTP.

La memoria de los JVM sube 110 MiB y 165 MiB. Ahí vive el agente, sus hilos y las colas de exportación. La CPU media de la ventana sube 10 y 15 puntos porcentuales. service-b crece más: atiende cada consulta de inventario y su JDBC. La ventana de `docker stats` dura 40 segundos e incluye el arranque del contenedor recién recreado, así que no es un perfil de estado estacionario de varios minutos.

La CPU del Collector bajó en la pasada instrumentada. No se toma como un ahorro: el valor de partida es bajo y la muestra es corta. La memoria del Collector sí sube, de 57 MiB a 71 MiB, que es coherente con recibir lotes.

Para bajar el costo sin apagar la visibilidad: intervalo de métricas por encima de 15 s, muestreo `parentbased_traceidratio` menor que 1 fuera del laboratorio, verbosidad `debug` del Collector en `basic`, y no abrir spans manuales alrededor del SQL. El batch del agente y el del Collector ya agrupan envíos.

Esta corrida no se repitió. Los JSON de origen quedan en `load-tests/results/` para poder recalcular.

10. Limitaciones y trabajo pendiente

Hecho y comprobado en local: compilación y `mvn test`, compose, pedido real que llama a service-b y a MySQL, traza con los dos servicios, logs JSON con el mismo `trace_id`, series en Prometheus, dashboard con datos de la carga, y el benchmark de esta sección.

Pendiente, y no simulado:

- `terraform apply` en GCP y en AWS. No hay credenciales. En esta máquina tampoco estaba el CLI de Terraform, así que `terraform validate` no se ejecutó. Los Collector de GCP y AWS sí pasaron `otelcol validate`.
- El clic visual de un exemplar de Prometheus hacia Tempo.
- Repetir el benchmark con más usuarios y más de una pasada. Los porcentajes de arriba no deben citarse como capacidad de producción.
- El p99 del panel de Grafana no debe citarse como resultado de k6. El resultado de k6 es la tabla de la sección 8.

11. Referencias

- OpenTelemetry, especificación de trazas, métricas y propagación W3C Trace Context.
- OpenTelemetry Java Agent 2.31.1 y API 1.65.0.
- OpenTelemetry Collector Contrib 0.162.0. Nombres vigentes `otlp_grpc` y `otlp_http`.

- Grafana, aprovisionamiento de datasources: `$$` conserva un `$` literal. Documentación de derived fields y `tracesToLogsV2`.
- Spring Boot 3.5.16, MySQL 8.4, Prometheus 3.4.1, Grafana 11.6.0, Tempo 2.8.1, Loki 3.4.2, k6 0.57.0.
- Documentos del propio proyecto: `docs/analisis-repositorio-docente.md`, `docs/arquitectura.md`, `docs/instrumentacion-opentelemetry.md`, `docs/collector.md`, `docs/correlacion-cross-signal.md`, `docs/benchmark-overhead.md`.